

Big Data Analytics

Sr. No	Details	Sign
1	Install, configure and run Hadoop and HDFS and explore HDFS.	
2	Implement word count / frequency programs using MapReduce	
3	Implement an MapReduce program that processes a weather dataset.	
4	Implement an application that stores big data in Hbase / MongoDB and manipulate it using R / Python	
5	Implement the program in practical 4 using Pig.	
6	Configure the Hive and implement the application in Hive.	
7	Write a program to illustrate the working of Jaql.	
8	Implement Decision tree classification techniques	
9	Implement SVM classification techniques	
10	REGRESSION MODEL Import a data from web storage. Name the dataset and now do Logistic Regression to find out relation between variables that are affecting the admission of a student in an institute based on his or her GRE score, GPA obtained and rank of the student. Also check the model is fit or not. require (foreign), require(MASS).	
11	MULTIPLE REGRESSION MODEL Apply multiple regressions, if data have a continuous independent variable. Apply on above dataset.	
12	CLASSIFICATION MODEL a. Install relevant package for classification. b. Choose classifier for classification problem. c. Evaluate the performance of classifier.	
13	CLUSTERING MODEL a. Clustering algorithms for unsupervised classification. b. Plot the cluster data using R visualizations.	

PRACTICAL NO. 1

Aim: Install, configure and run Hadoop and HDFS and explore HDFS.

Code:

```
# 1 Install Java 8 only
!apt-get update -qq
!apt-get install -y openjdk-8-jdk

# 2 Check Java version
!java -version

# 3 Download Hadoop 3.3.6
!wget -q https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
!tar -xzf hadoop-3.3.6.tar.gz
!mv hadoop-3.3.6 /usr/local/hadoop

# 4 Set environment variables
import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
os.environ["HADOOP_HOME"] = "/usr/local/hadoop"
os.environ["PATH"] += f":{os.environ['HADOOP_HOME']/bin}:{os.environ['HADOOP_HOME']/sbin}"

# 5 Configure core-site.xml
!sed -i '/<configuration>/a \
<property>\
<name>fs.defaultFS</name>\
<value>hdfs://localhost:9000</value>\
</property>' /usr/local/hadoop/etc/hadoop/core-site.xml

# 6 Configure hdfs-site.xml
!sed -i '/<configuration>/a \
<property>\
<name>dfs.replication</name>\
<value>1</value>\
</property>\
<property>\
<name>dfs.namenode.name.dir</name>\
<value>file:///usr/local/hadoop/hdfs/namenode</value>\
</property>\
<property>\
<name>dfs.datanode.data.dir</name>\
<value>file:///usr/local/hadoop/hdfs/datanode</value>\
</property>' /usr/local/hadoop/etc/hadoop/hdfs-site.xml

# 8 Create HDFS directories
!mkdir -p /usr/local/hadoop/hdfs/namenode
!mkdir -p /usr/local/hadoop/hdfs/datanode

# 9 Format the NameNode
!hdfs namenode -format -force

# 10 Start HDFS daemons
!hdfs --daemon start namenode
!hdfs --daemon start datanode
!hdfs --daemon start secondarynamenode

# 11 Verify with jps
!jps
```

Output:

```
/
SHUTDOWN_MSG: Shutting down NameNode at 65a9dd9e6a6c/172.28.0.12
*****/

1731 NameNode
1796 DataNode
1881 Jps
1851 SecondaryNameNode
```

PRACTICAL NO. 2

Aim: To implement and execute a MapReduce program for counting word frequencies from a dataset stored in HDFS (simulated in Google Colab) using Python's `mrjob` library.

Code:

```
# Step 1: Install required libraries
!pip install mrjob

# Step 2: Create sample data
print("Creating sample data...")
sample_text = """
The quick brown fox jumps over the lazy dog.
The dog sleeps while the fox jumps again.
MapReduce is a programming model for processing large datasets.
Hadoop implements MapReduce for distributed computing.
The quick brown fox is quick and brown.
Apache Hadoop is a framework that allows for the distributed processing of large data sets across clusters of computers.
MapReduce programs are parallel in nature and thus are very useful for performing large-scale data analysis.
The word count program is the "Hello World" of MapReduce.
"""

with open('/content/sample_data.txt', 'w') as f:
    f.write(sample_text)

print("Sample data created successfully!")
!cat /content/sample_data.txt

# Step 3: Create the MapReduce program
mapreduce_code = '''
from mrjob.job import MRJob
from mrjob.step import MRStep
import re
import sys

class MRWordCount(MRJob):

    def configure_args(self):
        super(MRWordCount, self).configure_args()
        self.add_passthru_arg('--min-word-length', type=int, default=1,
                               help='Minimum word length to count')

    def mapper(self, _, line):
        words = re.findall(r'\b\w+\b', line.lower())
        min_length = self.options.min_word_length
        for word in words:
            if len(word) >= min_length:
                yield (word, 1)

    def combiner(self, word, counts):
        yield (word, sum(counts))

    def reducer(self, word, counts):
        yield (word, sum(counts))

    def steps(self):
        return [

```

```

        MRStep(mapper=self.mapper,
                combiner=self.combiner,
                reducer=self.reducer),
    ]

if __name__ == '__main__':
    sys.argv = [arg for arg in sys.argv if not arg.startswith('-f')]
    MRWordCount.run()
...

with open('/content/wordcount.py', 'w') as f:
    f.write(mapreduce_code)

print("MapReduce program saved to /content/wordcount.py")

# Step 4: Run the MapReduce job
print("\nRunning MapReduce job...")
!python /content/wordcount.py /content/sample_data.txt --min-word-length 2 > /content/results.txt 2>&1

print("\nRESULTS:")
!cat /content/results.txt | tail -20

# Step 5: Extract and analyze word counts
print("\nEXTRACTED WORD COUNTS")
word_counts = []
with open('/content/results.txt', 'r') as f:
    for line in f:
        if '\t' in line and not line.startswith('no config') and not line.startswith('WARNING'):
            parts = line.strip().split('\t')
            if len(parts) == 2 and parts[1].isdigit():
                word_counts.append((parts[0], int(parts[1])))

if word_counts:
    word_counts.sort(key=lambda x: x[1], reverse=True)
    print(f"{'WORD':<20} {'COUNT':<10}")
    print("-" * 30)
    for word, count in word_counts[:20]:
        print(f"{word:<20} {count:<10}")
    print(f"\nTotal unique words: {len(word_counts)}")
    print(f"Total word occurrences: {sum(c for _, c in word_counts)}")
else:
    print("No word counts found.")

# Step 6: Analyze MapReduce phases
print("\nMAPREDUCE PHASE ANALYSIS")
with open('/content/results.txt', 'r') as f:
    log_content = f.read()

print("\n1. MAPPER PHASE:")
mapper_lines = [line for line in log_content.split('\n') if 'mapper' in line.lower() or 'map' in line.lower()]
if mapper_lines:
    for line in mapper_lines[:5]:
        print(f" {line.strip()}")
else:
    print(" Mapper phase executed (logs not shown)")

print("\n2. REDUCER PHASE:")
reducer_lines = [line for line in log_content.split('\n') if 'reducer' in line.lower() or 'reduce' in line.lower()]
if reducer_lines:
    for line in reducer_lines[:5]:
        print(f" {line.strip()}")
else:
    print(" Reducer phase executed (logs not shown)")

```



```
# Step 7: Visualization
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

if word_counts:
    df = pd.DataFrame(word_counts, columns=['word', 'count'])
    df = df.sort_values('count', ascending=False)

    fig, axes = plt.subplots(2, 2, figsize=(15, 12))

    # Top 15 bar chart
    top_15 = df.head(15)
    axes[0, 0].bar(range(len(top_15)), top_15['count'], color='skyblue')
    axes[0, 0].set_title('Top 15 Most Frequent Words')
    axes[0, 0].set_xticks(range(len(top_15)))
    axes[0, 0].set_xticklabels(top_15['word'], rotation=45, ha='right')

    # Pie chart top 8
    top_8 = df.head(8)
    axes[0, 1].pie(top_8['count'], labels=top_8['word'], autopct='%1.1f%%')
    axes[0, 1].set_title('Top 8 Word Distribution')

    # Histogram of frequencies
    axes[1, 0].hist(df['count'], bins=20, color='green', alpha=0.7)
    axes[1, 0].set_title('Distribution of Word Frequencies')
    axes[1, 0].set_yscale('log')

    # Cumulative distribution
    sorted_counts = df['count'].values
    cumulative = np.cumsum(sorted_counts) / np.sum(sorted_counts)
    axes[1, 1].plot(range(1, len(cumulative)+1), cumulative, 'r-')
    axes[1, 1].set_title('Cumulative Word Frequency')
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print("\nStatistics:")
    print(f"  Total unique words: {len(df)}")
    print(f"  Total occurrences: {df['count'].sum()}")
    print(f"  Average frequency: {df['count'].mean():.2f}")
    print(f"  Median frequency: {df['count'].median()}")
    hapax = df[df['count'] == 1]
    print(f"  Words appearing once: {len(hapax)} ({len(hapax)/len(df)*100:.1f}%)")
else:
    print("No data for visualization.")

# Step 8: Performance testing
print("\nPERFORMANCE TESTING")
import time
import os
```

```
def run_wordcount_and_time(input_file, min_word_length=2):
    output_file = f"/content/results_{os.path.basename(input_file)}.txt"
    start = time.time()
    !python /content/wordcount.py {input_file} --min-word-length {min_word_length} > {output_file} 2>&1
    end = time.time()
    word_counts = []
    with open(output_file, 'r') as f:
        for line in f:
            if '\t' in line and not line.startswith('no config') and not line.startswith('WARNING'):
                parts = line.strip().split('\t')
                if len(parts) == 2 and parts[1].isdigit():
                    word_counts.append((parts[0], int(parts[1])))
    return end - start, len(word_counts), sum(c for _, c in word_counts)

# Small dataset
time_small, unique_small, total_small = run_wordcount_and_time('/content/sample_data.txt')
print(f"Small dataset (0.5 KB): {time_small:.2f}s, {unique_small} unique, {total_small} total")

# Medium dataset
with open('/content/medium_data.txt', 'w') as f:
    f.write(sample_text * 10)
time_medium, unique_medium, total_medium = run_wordcount_and_time('/content/medium_data.txt')
print(f"Medium dataset (5 KB): {time_medium:.2f}s, {unique_medium} unique, {total_medium} total")

# Large dataset
with open('/content/large_data.txt', 'w') as f:
    f.write(sample_text * 100)
time_large, unique_large, total_large = run_wordcount_and_time('/content/large_data.txt')
print(f"Large dataset (50 KB): {time_large:.2f}s, {unique_large} unique, {total_large} total")

# Plot performance
plt.figure(figsize=(10,6))
sizes = ['Small (0.5KB)', 'Medium (5KB)', 'Large (50KB)']
times = [time_small, time_medium, time_large]
plt.bar(sizes, times, color=['green', 'orange', 'red'])
plt.xlabel('Dataset Size')
plt.ylabel('Execution Time (seconds)')
plt.title('MapReduce WordCount Performance')
for i, v in enumerate(times):
    plt.text(i, v+0.01, f'{v:.2f}s', ha='center')
plt.show()
```

Output:

```
--- EXTRACTED WORD COUNTS
WORD                                COUNT
-----
"the"                                8
"for"                                4
"is"                                 4
"mapreduce"                          4
"fox"                                 3
"large"                              3
"quick"                             3
"brown"                             3
"of"                                 3
"distributed"                        2
"dog"                                2
"hadoop"                             2
"jumps"                              2
"and"                                2
"are"                                2
"data"                               2
"processing"                         2
"framework"                          1
"hello"                              1
"implements"                         1

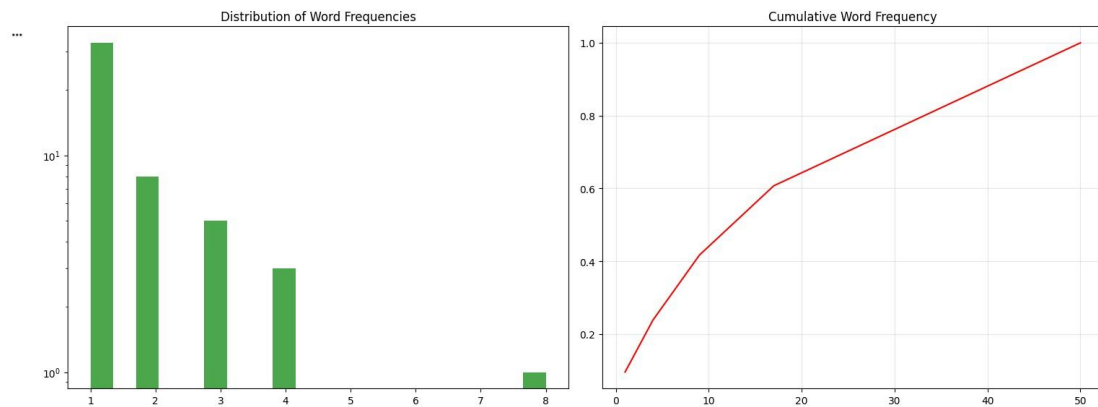
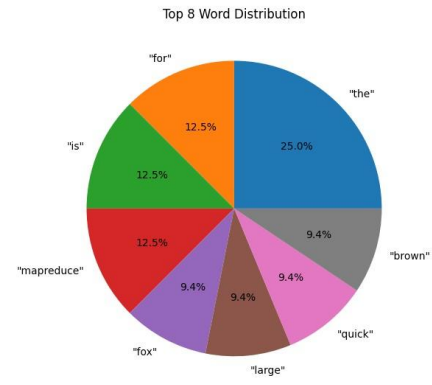
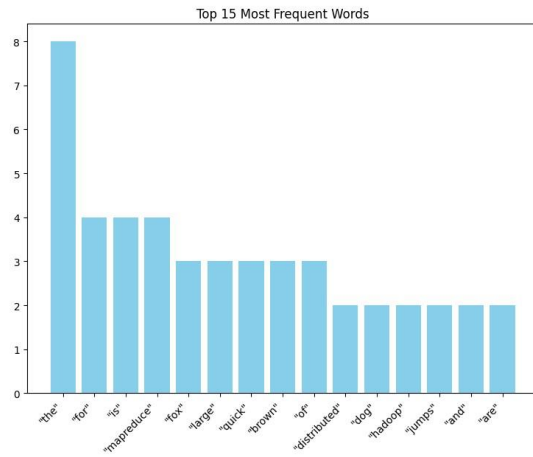
Total unique words: 50
Total word occurrences: 84

MAPREDUCE PHASE ANALYSIS

1. MAPPER PHASE:
  "mapreduce"    4

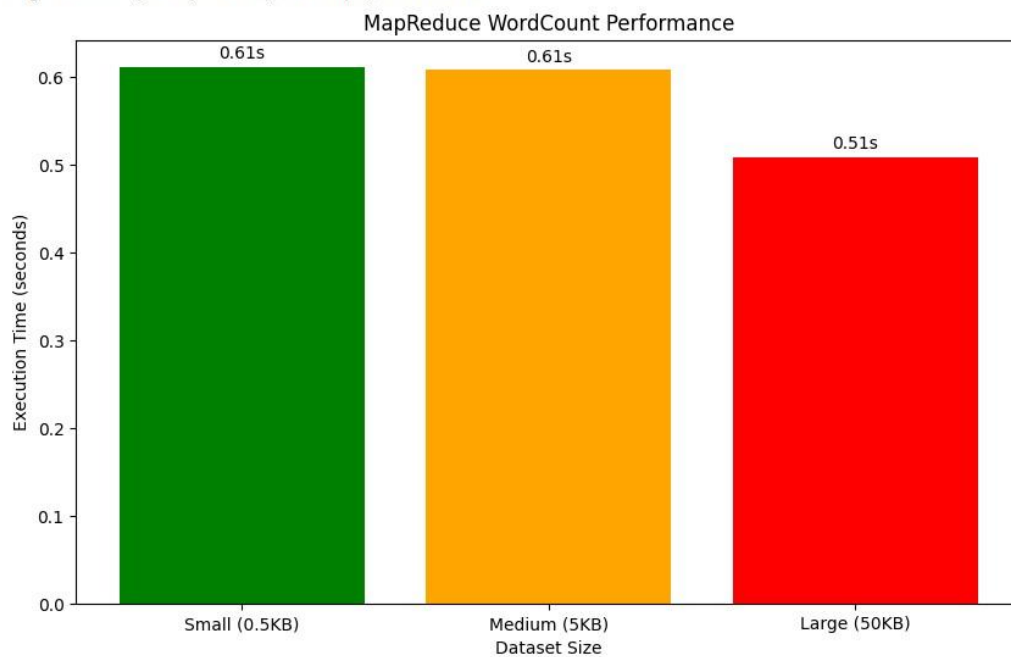
2. REDUCER PHASE:
  "mapreduce"    4
```

Big Data Analytics



Statistics:
 Total unique words: 50
 Total occurrences: 84
 Average frequency: 1.68
 Median frequency: 1.0
 Words appearing once: 33 (66.0%)

PERFORMANCE TESTING
 Small dataset (0.5 KB): 0.61s, 50 unique, 84 total
 Medium dataset (5 KB): 0.61s, 50 unique, 840 total
 Large dataset (50 KB): 0.51s, 50 unique, 8400 total



PRACTICAL NO. 3

Aim: Implement a MapReduce program that processes a Weather Dataset Description:

1. Upload the weather dataset to Google Colab.
2. The **Mapper program** reads the dataset and outputs (year, temperature).
3. The **Reducer program** calculates the **maximum temperature for each year**.
4. The final result displays the **highest temperature recorded in each year**.

Code:

Step 1: Sample Weather Dataset

```
Year, Temperature
2018,30
2018,32
2018,29
2019,35
2019,33
2019,31
2020,36
2020,38
2020,37
```

Mapper Code (mapper.py)

```
import sys

for line in sys.stdin:
    line = line.strip()

    if line.startswith("Year"):
        continue

    year, temp = line.split(",")

    print(f"{year}\t{temp}")
```

Reducer Code (reducer.py)

```
import sys

current_year = None
max_temp = -999
```

```
for line in sys.stdin:
    line = line.strip()

    year, temp = line.split("\t")
    temp = int(temp)

    if current_year == year:
        max_temp = max(max_temp,
temp)    else:        if current_year:
            print(f"{current_year}\t{max_temp}")
        current_year = year
        max_temp = temp

if current_year == year:
    print(f"{current_year}\t{max_temp}")
```

Google Colab Implementation Code

```
# Upload dataset from
google.colab import files
uploaded = files.upload()

# Install Hadoop
!apt-get install openjdk-8-jdk-headless -qq > /dev/null
!wget -q https://downloads.apache.org/hadoop/common/hadoop-
3.3.6/hadoop3.3.6.tar.gz
!tar xf hadoop-3.3.6.tar.gz

# Create Mapper File
%%writefile mapper.py
import sys

for line in sys.stdin:
    line=line.strip()    if
    line.startswith("Year"):
        continue
    year,temp=line.split(",")
    print(f"{year}\t{temp}")

# Create Reducer File
%%writefile reducer.py
import sys

current_year=None max_temp=-
999
```

```
for line in sys.stdin:
    line=line.strip()
    year,temp=line.split("\t")
    temp=int(temp)

    if current_year==year:
        max_temp=max(max_temp,temp)
    else:
        if current_year:
            print(f"{current_year}\t{max_temp}")
        current_year=year
        max_temp=temp

if current_year==year:
    print(f"{current_year}\t{max_temp}")

# Run MapReduce
!cat weather.csv | python mapper.py | sort | python reducer.py
```

Output:

```
2018 32
2019 35
2020 38
```

PRACTICAL NO. 4

Aim: Implement an application that stores big data in HBase and manipulates it using Python Code:

Step 1: Install Required Libraries

```
!pip install happybase
```

Step 2: Import

Libraries

```
import happybase
import pandas as pd
```

Step 3: Connect to HBase Server

```
connection = happybase.Connection('localhost')
print(connection.tables())
```

Step 4: Create HBase Table

```
connection.create_table(
    'student',
    {'info':
    dict()})
```

Step 5: Insert Data into Table

```
table = connection.table('student')
```

```
table.put('101', {
    'info:name': 'Amit',
    'info:course': 'Data Science',
    'info:marks': '85'
})
```

```
table.put('102', {
    'info:name': 'Neha',
    'info:course': 'AI',
    'info:marks': '90'
})
```

```
table.put('103', {
    'info:name': 'Rahul',
    'info:course': 'Big Data',
    'info:marks': '78'
})
```

Step 6: Retrieve Data from HBase

```
for key, data in table.scan():  
    print(key, data)
```

Step 7: Update Data

```
table.put('101', {'info:marks': '88'})
```

Step 8: Delete Data

```
table.delete('103')
```

Output:

Table Created Successfully

[b'student']

Inserted Records

```
b'101' {b'info:name': b'Amit', b'info:course': b'Data Science', b'info:marks':  
b'85'} b'102' {b'info:name': b'Neha', b'info:course': b'AI', b'info:marks': b'90'}  
b'103' {b'info:name': b'Rahul', b'info:course': b'Big Data', b'info:marks': b'78'}
```

After Update b'101' {b'info:name': b'Amit', b'info:course': b'Data Science',
b'info:marks': b'88'} **After Delete**

```
b'101' {...}  
b'102' {...}
```


PRACTICAL NO. 5

Aim: Implement an application that stores big data in HBase and manipulates it using Pig Code & Output:

Step 1: Install Java (Required for Hadoop)

```
!apt-get install openjdk-8-jdk-headless -qq > /dev/null
```

Step 2: Download and Extract Hadoop

```
!wget -q https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
!tar -xzf hadoop-3.3.6.tar.gz
```

Step 3: Set JAVA_HOME environment:

```
import os

# Set JAVA_HOME properly
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"

# Add Java to PATH
os.environ["PATH"] = os.environ["JAVA_HOME"] + "/bin:" + os.environ["PATH"]

# Verify Java
!java -version
```

```
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (build 1.8.0_482-8u482-ga~us1-0ubuntu1~22.04-b08)
OpenJDK 64-Bit Server VM (build 25.482-b08, mixed mode)
```

Step 4: Set Hadoop environment:

```
import os
os.environ["HADOOP_HOME"] = "/content/hadoop-3.3.6"
os.environ["PATH"] = os.environ["HADOOP_HOME"] + "/bin:" + os.environ["PATH"]
```

Step 5: Install Apache Pig

```
!wget -q https://downloads.apache.org/pig/pig-0.17.0/pig-0.17.0.tar.gz
!tar -xzf pig-0.17.0.tar.gz
```

Step 6: Create dataset

Student Dataset

Student_ID	Name	Course	Marks
101	Amit	Data Science	85
102	Neha	AI	90
103	Rahul	Big Data	78
104	Priya	Machine Learning	88
105	Kiran	Data Analytics	92

Step 7: Upload Dataset to HDFS (Simulation)

```
!mkdir -p input
!cp student_data.csv input/
!ls input
```

student_data.csv

Step 8: Create Pig Script

```
%%writefile student.pig
```

```
student_data = LOAD 'input/student_data.csv'
USING PigStorage(',')
AS (id:int, name:chararray, course:chararray, marks:int);

-- Display dataset
DUMP student_data;

-- Filter students with marks greater than 85
high_marks = FILTER student_data BY marks > 85;

DUMP high_marks;
```

Writing student.pig

Step 6: Run Pig Script

```
!pig-0.17.0/bin/pig student.pig
```

*** Success!

```
Job Stats (time in seconds):
JobId  Maps    Reduces MaxMapTime    MinMapTime    AvgMapTime    MedianMapTime    MaxReduceTime
job_local195973622_0001 1      0      n/a      n/a      n/a      n/a      0      0
```

```
Input(s):
Successfully read 5 records from: "file:///content/input/student_data.csv"
```

```
Output(s):
Successfully stored 5 records in: "file:/tmp/temp-451465548/tmp-1826667977"
```

```
Counters:
Total records written : 5
Total bytes written : 0
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
```

```
2020-04-08 17:50:14,824 [main] INFO org.ap
```

```
(101,Amit,Data Science,85)
(102,Neha,AI,90)
(103,Rahul,Big Data,78)
(104,Priya,Machine Learning,88)
```

```
(102,Neha,AI,90)
```

```
(104,Priya,Machine Learning,88)
```

```
(105,Kiran,Data Analytics,92)
```

```
2020-04-08 17:50:20,977 [main] INFO org.apache.pig.Main - Pig script completed in 16 seconds and 23 milliseconds (16023 ms)
```

Step 7:

```
# Simulated HBase Storage (Using Python Dictionary)
import pandas as pd

data = pd.read_csv("student_data.csv", header=None)
data.columns = ["ID", "Name", "Course", "Marks"]

# Simulated HBase table
hbase_table = {}

for index, row in data.iterrows():
    hbase_table[row["ID"]] = {
        "info:name": row["Name"],
        "info:course": row["Course"],
        "info:marks": row["Marks"]
    }

print(hbase_table)

{
101: {'info:name': 'Amit', 'info: course': 'Data Science', 'info:marks': 85},
102: {'info:name': 'Neha', 'info:course': 'AI', 'info:marks': 90},
103: {'info:name': 'Rahul', 'info:course': 'Big Data', 'info:marks': 78},
104: {'info:name': 'Priya', 'info:course': 'Machine Learning', 'info:marks': 88},
105: {'info:name': 'Kiran', 'info:course': 'Data Analytics', 'info:marks': 92}
}
```

PRACTICAL NO. 6

Aim: Configure the Hive and implement the application using Hive

Code:

Step 1: Create Dataset

```
%%writefile student_data.csv
101,Amit,Data Science,85
102,Neha,AI,90
103,Rahul,Big Data,78
104,Priya,Machine Learning,88
105,Kiran,Data Analytics,92
```

Step 2: Install Java

```
!apt-get install openjdk-8-jdk-headless -qq > /dev/null
```

Step 3: Download and Extract Hadoop

```
!wget -q https://downloads.apache.org/hadoop/common/hadoop-
3.3.6/hadoop3.3.6.tar.gz
!tar -xzf hadoop-3.3.6.tar.gz
```

Step 4: Download and Extract Hive

```
!wget -q https://downloads.apache.org/hive/hive-3.1.3/apache-hive-
3.1.3bin.tar.gz
!tar -xzf apache-hive-3.1.3-bin.tar.gz
```

Step 5: Create Hive Table Hive

Query:

```
CREATE TABLE student(
id INT,
name STRING, course
STRING,
marks INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Step 6: Load Data into Hive Table

```
LOAD DATA LOCAL INPATH 'student_data.csv'
INTO TABLE student;
```

Step 7: Display Table Data

```
SELECT * FROM student;
```

Step 8: Query Students with Marks > 85

SELECT * FROM student

WHERE marks > 85; **Output:**

Table Data

101	Amit	Data Science	85
102	Neha	AI	90
103	Rahul	Big Data	78
104	Priya	Machine Learning	88
105	Kiran	Data Analytics	92

Filtered Output

102	Neha	AI	90
104	Priya	Machine Learning	88
105	Kiran	Data Analytics	92

PRACTICAL NO. 7

Write a program to illustrate the working of Jaql.

Code:

```
# Simulating Jaql operations using Python
import json
import pandas as pd

# Sample JSON data
data = [
    {"name": "Amit", "marks": 85, "course": "DS"},
    {"name": "Sneha", "marks": 92, "course": "AI"},
    {"name": "Rahul", "marks": 70, "course": "DS"},
    {"name": "Pooja", "marks": 60, "course": "AI"}
]

# Convert to DataFrame df
df = pd.DataFrame(data)

# Display data
print("Original Data:")
print(df)

# Filter students with marks > 75
filtered = df[df["marks"] > 75]
print("\nFiltered Data (marks > 75):")
print(filtered)

# Transformation (add grade column)
df["grade"] = df["marks"].apply(lambda x: "A" if x > 80 else "B")
print("\nTransformed Data:")
print(df)
```

```
# Aggregation (average marks) avg_marks  
= df["marks"].mean() print("\nAverage  
Marks:", avg_marks)
```

Output:

```
*** Original Data:  
   name  marks course  
0  Amit    85     DS  
1  Sneha   92     AI  
2  Rahul   70     DS  
3  Pooja   60     AI  
  
Filtered Data (marks > 75):  
   name  marks course  
0  Amit    85     DS  
1  Sneha   92     AI  
  
Transformed Data:  
   name  marks course grade  
0  Amit    85     DS     A  
1  Sneha   92     AI     A  
2  Rahul   70     DS     B  
3  Pooja   60     AI     B  
  
Average Marks: 76.75
```

PRACTICAL NO. 8

Implement Decision tree classification techniques

Description:

Step 1: The package "party" has the function ctree() which is used to create and analyze decision tree.

```
> install.packages("party")
```

Step 2: Load the party package. It will automatically load other# dependent packages Print some records from data set readingSkills

```
> library("party")
> print(head(readingskills))
  nativespeaker age shoesize    score
1          yes   5  24.83189 32.29385
2          yes   6  25.95238 36.63105
3          no  11  30.42170 49.60593
4          yes   7  28.66450 40.28456
5          yes  11  31.88207 55.46085
6          yes  10  30.07843 52.83124
>
```

Step 3 : Call function ctree to build a decision tree. The first parameter is a formula, which defines a target variable and a list of independent variables.

```
> iris_ctree <- ctree(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width, data=iris)
> print(iris_ctree)
```

Conditional inference tree with 4 terminal nodes

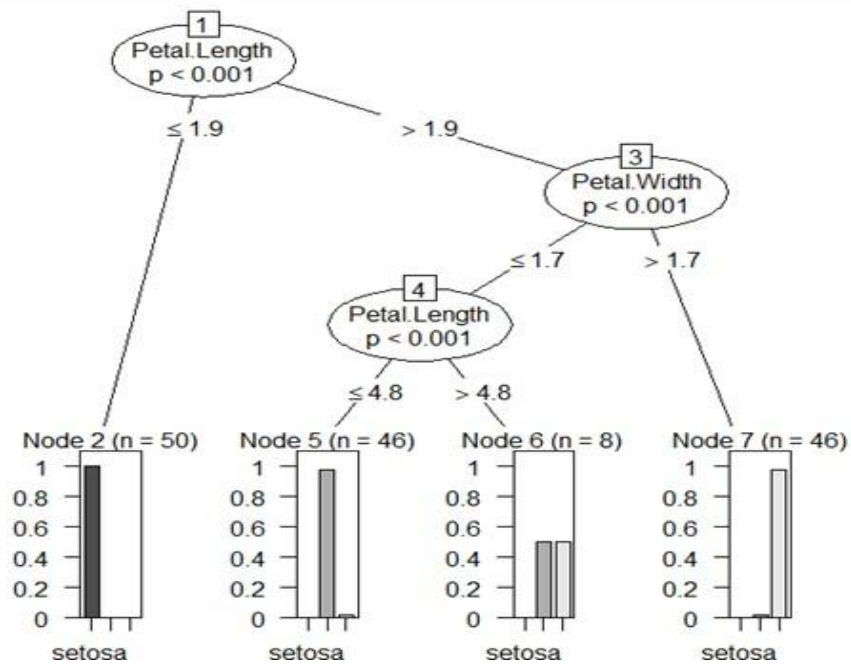
Response: Species

Inputs: Sepal.Length, Sepal.Width, Petal.Length, Petal.Width

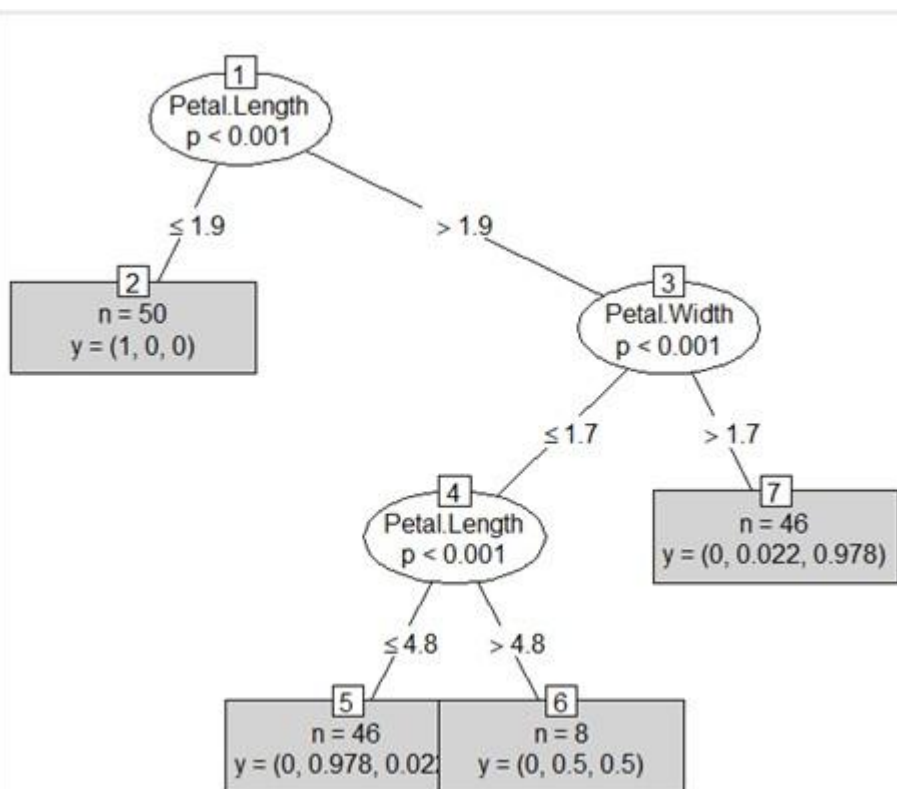
Number of observations: 150

```
1) Petal.Length <= 1.9; criterion = 1, statistic = 140.264
  2)* weights = 50
1) Petal.Length > 1.9
  3) Petal.Width <= 1.7; criterion = 1, statistic = 67.894
    4) Petal.Length <= 4.8; criterion = 0.999, statistic = 13.865
      5)* weights = 46
    4) Petal.Length > 4.8
      6)* weights = 8
    3) Petal.Width > 1.7
      7)* weights = 46
> plot(iris_ctree)
```


Output:



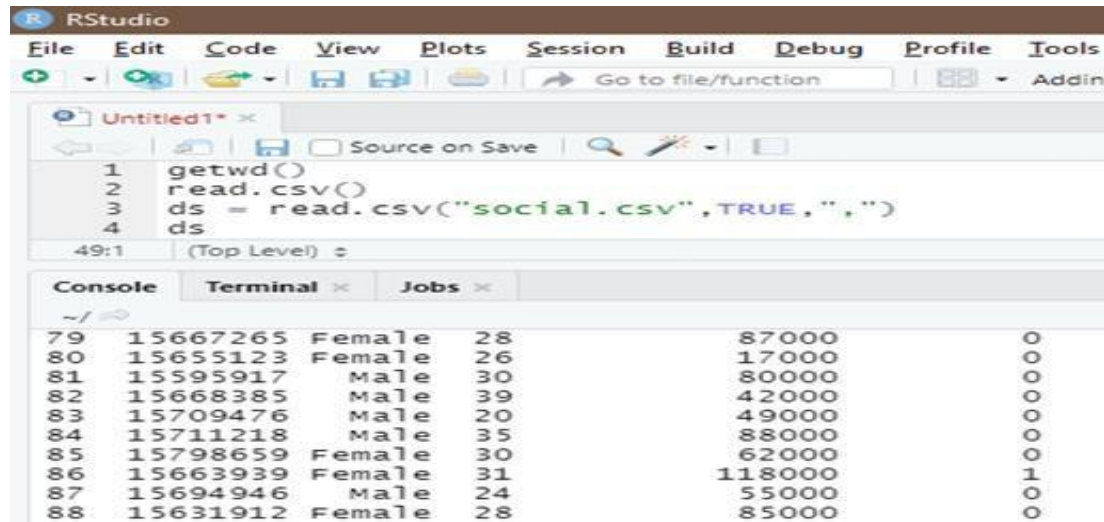
```
> plot(iris_ctree, type="simple")
```



PRACTICAL NO. 9

Implement SVM classification techniques Code:

Step 1: Importing the dataset



```

1 getwd()
2 read.csv()
3 ds = read.csv("social.csv", TRUE, ",", "")
4 ds

```

79	15667265	Female	28		87000	0
80	15655123	Female	26		17000	0
81	15595917	Male	30		80000	0
82	15668385	Male	39		42000	0
83	15709476	Male	20		49000	0
84	15711218	Male	35		88000	0
85	15798659	Female	30		62000	0
86	15663939	Female	31		118000	1
87	15694946	Male	24		55000	0
88	15631912	Female	28		85000	0

Step 2: Selecting columns 3-5

```

> ds = ds[3:5]
> ds[3:5]
Error in `[.data.frame'`(ds, 3:5) : undefined columns selected
> ds

```

	Age	EstimatedSalary	Purchased
1	19	19000	0
2	35	20000	0
3	26	43000	0
4	27	57000	0
5	19	76000	0
6	27	58000	0
7	27	84000	0
8	32	150000	1
9	25	33000	0
10	35	65000	0
11	26	80000	0
12	26	52000	0

Step 3: install package

```
> install.packages("caTools")
```

Step 4: Splitting the dataset

```

> library(caTools)
> set.seed(123)
> split = sample.split(ds$Purchased, splitRatio = 0.75)
> training_set = subset(ds, split == TRUE)
> test_set = subset(ds, split == FALSE)
> ds

```

	Age	EstimatedSalary	Purchased
1	19	19000	0
2	35	20000	0
3	26	43000	0
4	27	57000	0
5	19	76000	0
6	27	58000	0
7	27	84000	0
8	32	150000	1
9	25	33000	0
10	35	65000	0

Step 5: Feature Scaling

```

332  48      119000      1
333  42      65000      0
[ reached 'max' / getOption("max.print") -- omitted 67 rows ]
> test_set[-3] = scale(test_set[-3])
> training_set[-3] = scale(training_set[-3])
> test_set[-3] = scale(test_set[-3])
> test_set[-3]
      Age EstimatedSalary
2   -0.30419063      -1.51354339
4   -1.05994374      -0.32456026
5   -1.81569686       0.28599864
9   -1.24888202      -1.09579256
12  -1.15441288      -0.48523366
18   0.64050076      -1.32073531
19   0.73496990      -1.25646596
20   0.92390818      -1.22433128
22   0.82943904      -0.58163769
29  -0.87100546      -0.77444577
32  -1.05994374       2.24621408
34  -0.96547460      -0.74231109
35  -1.05994374       0.73588415
38  -0.77653633      -0.58163769
45  -0.96547460       0.54307608
46  -1.43782030      -1.51354339

```

Step 6: Fitting SVM to the training set

```

> install.packages('e1071')

> library(e1071)
> classifier = svm(formula = Purchased ~ .,
+                  data = training_set,
+                  type = 'c-classification',
+                  kernel = 'linear')
> classifier

Call:
svm(formula = Purchased ~ ., data = training_set, type = "c-classification",
    kernel = "linear")

Parameters:
  SVM-Type:  c-classification
 SVM-kernel: linear
      cost:  1

Number of Support Vectors: 116

```

Step 7: Predicting the test set result

```

> y_pred = predict(classifier, newdata = test_set[-3])
> y_pred
 2  4  5  9 12 18 19 20 22 29 32 34 35 38 45 46 48 52 66
0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
69 74 75 82 84 85 86 87 89 103 104 107 108 109 117 124 126 127 131
0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0
134 139 148 154 156 159 162 163 170 175 176 193 199 200 208 213 224 226 228
0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  1  1  0  1
229 230 234 236 237 239 241 255 264 265 266 273 274 281 286 292 299 302 305
0  1  1  1  1  0  1  1  1  0  1  1  1  1  1  0  1  1  1  0
307 310 316 324 326 332 339 341 343 347 353 363 364 367 368 369 372 373 380
1  0  0  0  0  1  0  1  0  1  1  0  1  1  1  0  1  0  1
383 389 392 395 400
1  0  0  0  0
Levels: 0 1

```

```

> cm = table(test_set[, 3], y_pred)
> cm
      y_pred
      0  1
0  57  7
1 13 23

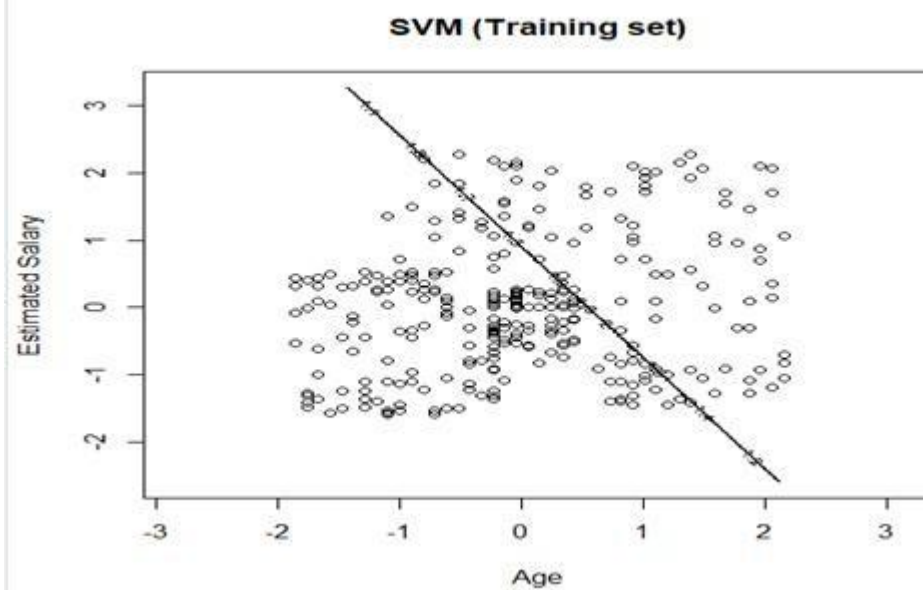
```

Step 8: Visualizing the Training set results

```
> set = training_set
> x1 = seq(min(set[, 1]) - 1, max(set[, 1]) + 1, by = 0.01)
> x2 = seq(min(set[, 2]) - 1, max(set[, 2]) + 1, by = 0.01)
```



```
> grid_set = expand.grid(x1, x2)
> colnames(grid_set) = c('Age', 'EstimatedSalary')
> y_grid = predict(classifier, newdata = grid_set)
> plot(set[, -3],
+       main = 'SVM (Training set)',
+       xlab = 'Age', ylab = 'Estimated salary',
+       xlim = range(x1), ylim = range(x2))
```



Output:

```
> contour(X1, X2, matrix(as.numeric(y_grid), length(X1), length(X2)), add = TRUE)  
> points(grid_set, pch = '.', col = ifelse(y_grid == 1, 'coral1', 'aquamarine'))  
> points(set, pch = 21, bg = ifelse(set[, 3] == 1, 'green4', 'red3'))
```



PRACTICAL NO. 10

Aim: To implement REGRESSION MODEL (Logistical Regression) Code:

```
#Step 1: import required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
▶ #Step 2: Import Dataset from Web Storage
url = "https://stats.idre.ucla.edu/stat/data/binary.csv"

data = pd.read_csv(url)

print("First 5 Records of Dataset")
print(data.head())
```

```
... First 5 Records of Dataset
   admit  gre  gpa  rank
0      0  380  3.61    3
1      1  660  3.67    3
2      1  800  4.00    1
3      1  640  3.19    4
4      0  520  2.93    4
```

```
) #Step 3: Dataset Information
print("\nDataset Information")
print(data.info())

print("\nStatistical Summary")
print(data.describe())

print("\nMissing Values")
print(data.isnull().sum())
```

```
Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   admit   400 non-null         int64
1   gre     400 non-null         int64
2   gpa     400 non-null         float64
3   rank    400 non-null         int64
dtypes: float64(1), int64(3)
memory usage: 12.6 KB
None

Statistical Summary
      admit      gre      gpa      rank
count  400.000000  400.000000  400.000000  400.000000
mean    0.317500  587.700000  3.389900  2.485000
std     0.466087  115.516536  0.380567  0.944466
min     0.000000  220.000000  2.260000  1.000000
25%     0.000000  520.000000  3.130000  2.000000
50%     0.000000  580.000000  3.395000  2.000000
75%     1.000000  660.000000  3.670000  3.000000
max     1.000000  800.000000  4.000000  4.000000

Missing Values
admit    0
gre      0
gpa      0
rank     0
dtype: int64
```

```
#Step 4: Convert Rank to Dummy Variables
rank_dummies = pd.get_dummies(data['rank'], prefix='rank', drop_first=True)

data = pd.concat([data, rank_dummies], axis=1)

data = data.drop('rank', axis=1)

print("\nDataset after creating dummy variables")
print(data.head())
```

```
Dataset after creating dummy variables
   admit  gre  gpa  rank_2  rank_3  rank_4
0      0  380  3.61   False    True   False
1      1  660  3.67   False    True   False
2      1  800  4.00   False   False   False
3      1  640  3.19   False   False    True
4      0  520  2.93   False   False    True
```

```
#Step 5: Define Dependent and Independent Variables
X = data[['gre', 'gpa', 'rank_2', 'rank_3', 'rank_4']]
y = data['admit']

# Add constant
X = sm.add_constant(X)

# Convert boolean columns to integers (0 and 1) for statsmodels
X[['rank_2', 'rank_3', 'rank_4']] = X[['rank_2', 'rank_3', 'rank_4']].astype(int)
```

```
# Step 6: Build Logistic Regression Model
logit_model = sm.Logit(y, X)

result = logit_model.fit()

print("\nLogistic Regression Summary")
print(result.summary())
```

```
Optimization terminated successfully.
      Current function value: 0.573147
      Iterations 6
```

Logistic Regression Summary

```

                                Logit Regression Results
=====
Dep. Variable:                  admit    No. Observations:                   400
Model:                            Logit    Df Residuals:                      394
Method:                           MLE     Df Model:                            5
Date:                Wed, 11 Mar 2026    Pseudo R-squ.:                   0.08292
Time:                   13:55:19         Log-Likelihood:                  -229.26
converged:                        True    LL-Null:                          -249.99
Covariance Type:            nonrobust    LLR p-value:                       7.578e-08
=====
               coef      std err          z      P>|z|      [0.025      0.975]
-----
const         -3.9900         1.140     -3.500     0.000     -6.224     -1.756
gre             0.0023         0.001      2.070     0.038      0.000      0.004
gpa             0.8040         0.332      2.423     0.015      0.154      1.454
rank_2         -0.6754         0.316     -2.134     0.033     -1.296     -0.055
rank_3         -1.3402         0.345     -3.881     0.000     -2.017     -0.663
rank_4         -1.5515         0.418     -3.713     0.000     -2.370     -0.733
=====
```

```
# Step 7: Predict Admission Probability
predictions = result.predict(X)

data['predicted_probability'] = predictions

print("\nPredicted Probabilities")
print(data[['admit', 'predicted_probability']].head())
```

	admit	predicted_probability
0	0	0.172627
1	1	0.292175
2	1	0.738408
3	1	0.178385
4	0	0.118354

```
# Step 8: Convert Probability to Binary Class
data['predicted_admit'] = (data['predicted_probability'] >= 0.5).astype(int)

print("\nPredicted Admission")
print(data[['admit', 'predicted_admit']].head())
```

	admit	predicted_admit
0	0	0
1	1	0
2	1	1
3	1	0
4	0	0

```
# Step 9: Evaluate Model Performance
cm = confusion_matrix(y, data['predicted_admit'])

accuracy = accuracy_score(y, data['predicted_admit'])

print("\nConfusion Matrix")
print(cm)

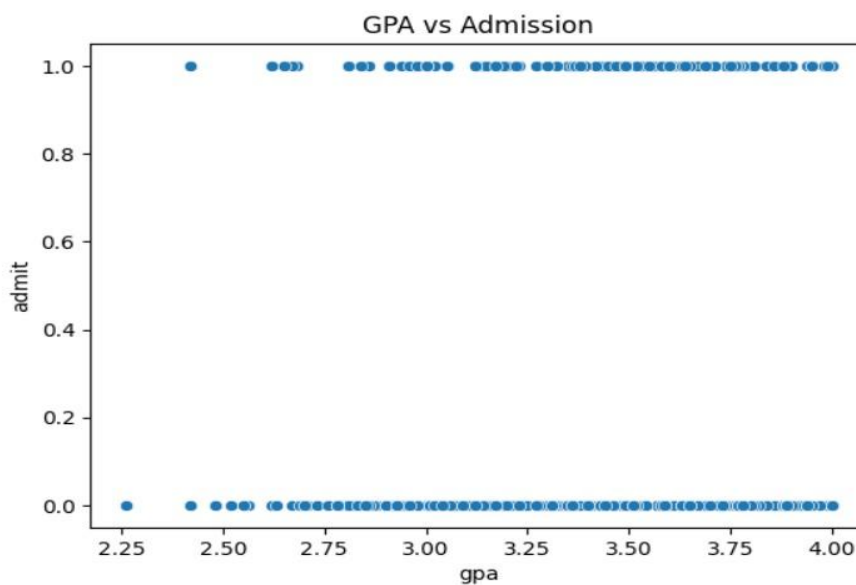
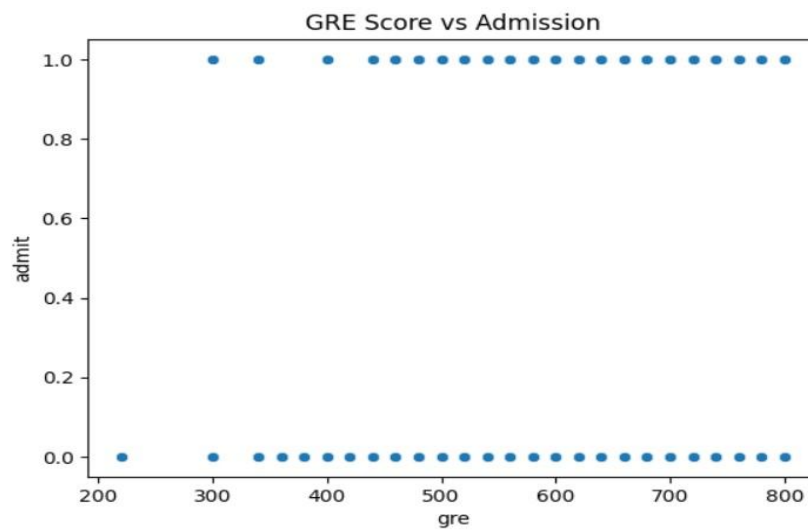
print("\nModel Accuracy:", accuracy)
```

```
Confusion Matrix
[[254  19]
 [ 97  30]]
```

```
Model Accuracy: 0.71
```

```
# Step 10: Visualization
plt.figure()
sns.scatterplot(x='gre', y='admit', data=data)
plt.title("GRE Score vs Admission")
plt.show()

plt.figure()
sns.scatterplot(x='gpa', y='admit', data=data)
plt.title("GPA vs Admission")
plt.show()
```


Output:

```
# Step 11: Conclusion
print("\nConclusion:")
print("Logistic Regression model identifies how GRE, GPA, and Rank influence admission probability.")
print("Model fitness can be checked using Pseudo R2 and LLR p-value from the summary output.")
```

Conclusion:
 Logistic Regression model identifies how GRE, GPA, and Rank influence admission probability.
 Model fitness can be checked using Pseudo R² and LLR p-value from the summary output.

PRACTICAL NO. 11

Aim: To implement Multiple Regression model Code:

```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Step 2: Import Dataset from Web Storage (FIXED URL)
url = "https://stats.idre.ucla.edu/stat/data/binary.csv"
data = pd.read_csv(url)

print(data.head())
```

```
•   admit  gre  gpa  rank
0      0   380  3.61     3
1      1   660  3.67     3
2      1   800  4.00     1
3      1   640  3.19     4
4      0   520  2.93     4
```

```
# Step 3: Define Variables and handle 'rank' as categorical
y = data['admit'] # Dependent variable

# Create dummy variables for 'rank' and drop the first one to avoid multicollinearity
rank_dummies = pd.get_dummies(data['rank'], prefix='rank', drop_first=True)

# Combine original predictors (gre, gpa) with the new rank dummy variables
X = data[['gre', 'gpa']]
X = pd.concat([X, rank_dummies], axis=1)

# Add constant (intercept) to the independent variables
X = sm.add_constant(X)

# Convert boolean dummy variables to integers (0 and 1) for statsmodels
X[['rank_2', 'rank_3', 'rank_4']] = X[['rank_2', 'rank_3', 'rank_4']].astype(int)

# Step 4: Apply Logistic Regression (Changed from Multiple Linear Regression)
model = sm.Logit(y, X).fit()
```

```
Optimization terminated successfully.
      Current function value: 0.573147
      Iterations 6
```

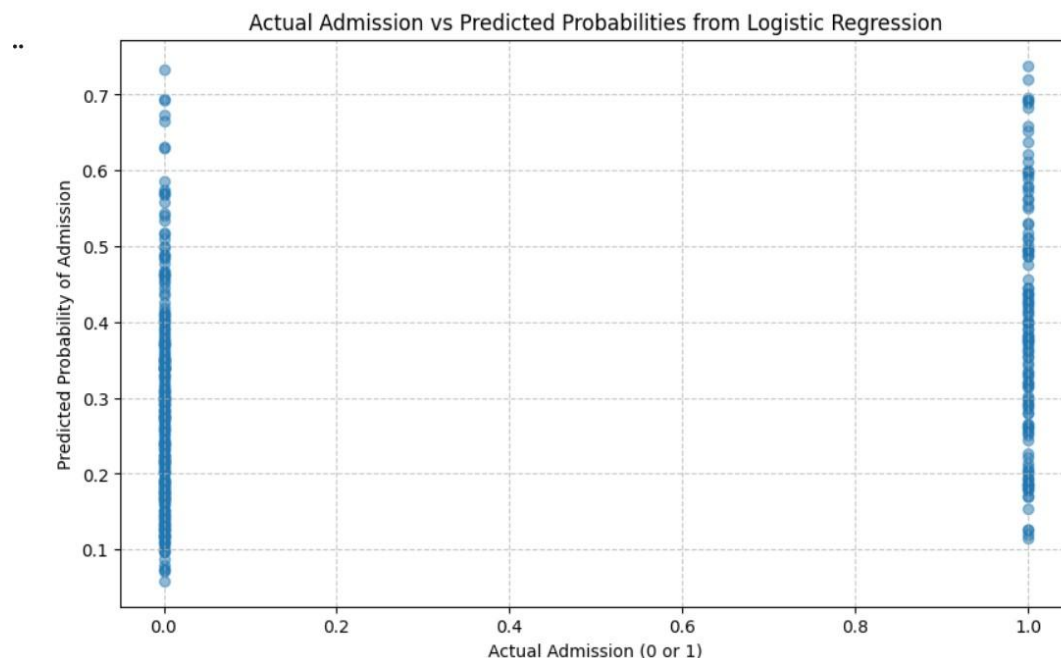
```
# Step 5: Print Model Summary
print(model.summary())
```

Logit Regression Results						
Dep. Variable:	admit	No. Observations:	400			
Model:	Logit	Df Residuals:	394			
Method:	MLE	Df Model:	5			
Date:	Wed, 11 Mar 2026	Pseudo R-squ.:	0.08292			
Time:	14:05:06	Log-Likelihood:	-229.26			
converged:	True	LL-Null:	-249.99			
Covariance Type:	nonrobust	LLR p-value:	7.578e-08			
	coef	std err	z	P> z	[0.025	0.975]
const	-3.9900	1.140	-3.500	0.000	-6.224	-1.756
gre	0.0023	0.001	2.070	0.038	0.000	0.004
gpa	0.8040	0.332	2.423	0.015	0.154	1.454
rank_2	-0.6754	0.316	-2.134	0.033	-1.296	-0.055
rank_3	-1.3402	0.345	-3.881	0.000	-2.017	-0.663
rank_4	-1.5515	0.418	-3.713	0.000	-2.370	-0.733

```
# Step 6: Predict probabilities
predictions = model.predict(X)
```

```
# Step 7: Plot Actual Admission vs Predicted Probabilities
plt.figure(figsize=(10, 6))
plt.scatter(y, predictions, alpha=0.5)
plt.xlabel("Actual Admission (0 or 1)")
plt.ylabel("Predicted Probability of Admission")
plt.title("Actual Admission vs Predicted Probabilities from Logistic Regression")
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Output:



PRACTICAL NO. 12

Aim: To implement Classification Model Code:

```
#classification model

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.datasets import load_iris

# Load Dataset
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['Target'] = data.target
# Split Data into Training and Testing Sets
X = df.drop(columns=['Target'])
y = df['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize and Train Classifier
classifier = RandomForestClassifier(n_estimators=100, random_state=42)
classifier.fit(X_train, y_train)
# Make Predictions
y_pred = classifier.predict(X_test)
# Evaluate the Model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
# Print Results
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", conf_matrix)
print("Classification Report:\n", report)
```

Output:

```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

     0           1.00      1.00      1.00         10
     1           1.00      1.00      1.00          9
     2           1.00      1.00      1.00         11

 accuracy          1.00
macro avg          1.00      1.00      1.00         30
weighted avg          1.00      1.00      1.00         30
```

PRACTICAL NO. 13

Aim: To implement clustering Model (K-Means clustering)

Code and Output:

Step 1: Apply kmeans to newiris, and store the clustering result in kc. The cluster number is set to 3.

```
> newiris <- iris
> newiris$species <- NULL
> (kc <- kmeans(newiris, 3))
K-means clustering with 3 clusters of sizes 38, 62, 50

Cluster means:
  Sepal.Length Sepal.width Petal.Length Petal.width
1    6.850000    3.073684    5.742105    2.071053
2    5.901613    2.748387    4.393548    1.433871
3    5.006000    3.428000    1.462000    0.246000

Clustering vector:
[1] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
[35] 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2
[69] 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2
[103] 1 1 1 1 2 1 1 1 1 1 1 2 2 1 1 1 1 2 1 2 1 2 1 1 1 2 2 1 1 1 1 2 1 1
[137] 1 1 2 1 1 1 2 1 1 1 2 1 1 2

within cluster sum of squares by cluster:
[1] 23.87947 39.82097 15.15100
(between_SS / total_SS = 88.4 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
```

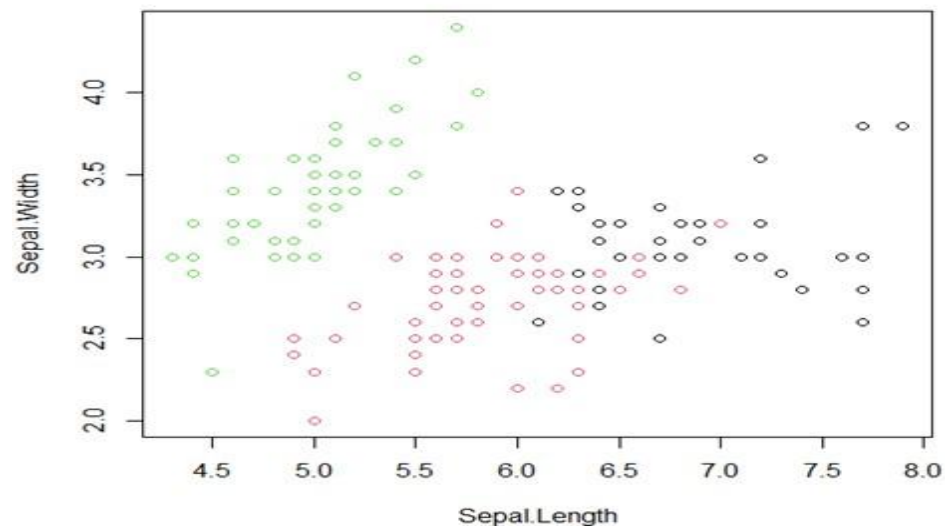
Step 2: Compare the Species label with the clustering result

```
> table(iris$species, kc$cluster)

      1  2  3
setosa  0  0 50
versicolor 2 48  0
virginica 36 14  0
```

Step 3 : Plot the clusters and their centres. Note that there are four dimensions in the data and that only the first two dimensions are used to draw the plot below.

```
> plot(newiris[c("Sepal.Length", "Sepal.width")], col=kc$cluster)
```

Step 4: Some black points close to the green centre (asterisk) are actually closer to the black centre in the four dimensional space.

```
> points(kc$centers[,c("sepal.Length", "sepal.Width")], col=1:3, pch=8, cex=2)
```

